

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 5: Construct interoperable web services by leveraging JAX-RPC, WSDL, SOAP, and XML Schema for seamless data exchange across distributed systems. (L4, L5)

Assessment Tool : Industry Problem Task

Weightage: 40%

QUESTIONS:

Total Marks: 30

1. A hospital management system requires a **SOAP-based web service** to manage patient information such as registration, appointment scheduling, and medical records. The system must ensure secure, structured, and interoperable communication between different hospital departments.

Design a **SOAP-based web service** for the hospital system. Explain the structure of SOAP messages (Envelope, Header, Body), define the operations involved (e.g., add patient, get records), and illustrate how data is exchanged between client and server. Justify the use of SOAP in this scenario considering reliability and security.

2. An enterprise application needs to expose its web services so that external systems can understand how to interact with them. The organization plans to use **WSDL (Web Services Description Language)** to describe the service.

Design the **WSDL structure** for a web service that provides operations such as retrieving and updating data. Explain the key components of WSDL (types, message, portType, binding, service), and illustrate how they define the service interface and communication details. Provide a clear structure and justify its importance in service integration.

3. A company is experiencing issues in its SOAP-based communication system where requests are not processed correctly, and responses are either delayed or incorrect.

Analyze the SOAP communication process and identify possible issues such as message formatting errors, incorrect namespaces, or endpoint problems. Propose a debugging approach to resolve these issues, including validation of SOAP messages and error handling techniques. Explain how the corrected system ensures reliable communication.

Code of Conduct:

I N . L o k a B a n d h a v a / 1 9 2 3 2 4 1 6 7 __ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

**RUBRICS FOR CALCULATION:**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

QUESTION 1: SOAP-Based Hospital Management Web Service

1. Introduction

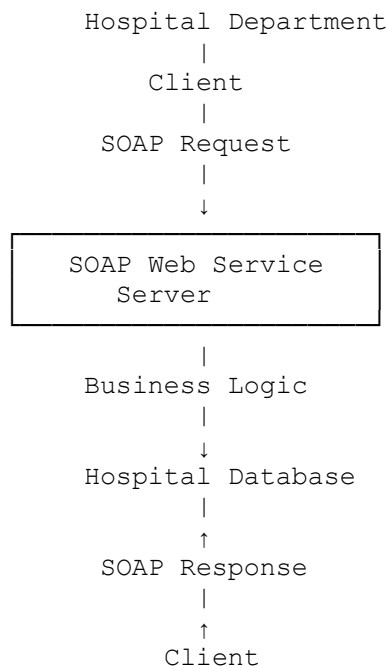
A hospital management system handles important information such as patient registration, appointment scheduling, and medical records. Since different departments such as reception, outpatient departments, laboratories, pharmacy, and billing may use different applications, they need a common mechanism to exchange information.

A **SOAP (Simple Object Access Protocol)** based web service can provide a structured and interoperable communication mechanism between these departments. SOAP uses XML messages to exchange information between a client and a server.

The proposed system provides operations such as:

- Add Patient
- Get Patient Details
- Schedule Appointment
- Get Medical Records
- Update Medical Records

The basic communication architecture is:

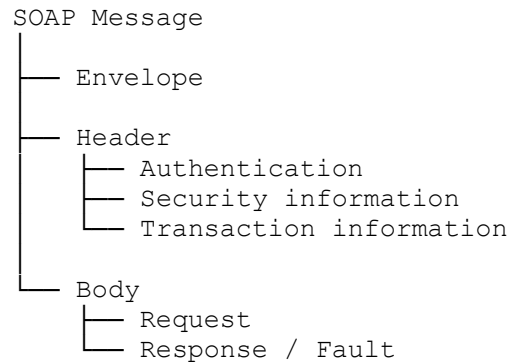


2. SOAP Message Structure

A SOAP message consists mainly of three important parts:

1. **Envelope**
2. **Header**
3. **Body**

Structure



2.1 SOAP Envelope

The **Envelope** is the root element of every SOAP message. It identifies the message as a SOAP message.

Example:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"

  <soap:Header>
    ...
  </soap:Header>

  <soap:Body>
    ...
  </soap:Body>

</soap:Envelope>
```

The envelope defines the beginning and end of the SOAP message.

3. SOAP Header

The **Header** contains additional information that is not directly part of the main business data.

For a hospital system, it can contain:

- Authentication information
- User credentials
- Security tokens
- Transaction identifiers
- Session information

Example:

```
<soap:Header>
  <Security>
    <Username>hospital_user</Username>
    <Token>ABC123XYZ</Token>
  </Security>
</soap:Header>
```

This helps the system authenticate the department or application making the request.

4. SOAP Body

The **Body** contains the actual request or response.

For example, a request to add a patient may contain:

```
<soap:Body>
  <AddPatient>
    <Patient>
      <PatientID>P1001</PatientID>
      <Name>Arun Kumar</Name>
      <Age>35</Age>
      <Gender>Male</Gender>
      <Phone>9876543210</Phone>
    </Patient>
  </AddPatient>
</soap:Body>
```

The server processes this request and sends a response.

Example response:

```
<soap:Body>
  <AddPatientResponse>
    <PatientID>P1001</PatientID>
    <Status>Patient Added Successfully</Status>
  </AddPatientResponse>
</soap:Body>
```

5. Operations of the Hospital SOAP Service

The proposed web service can provide the following operations.

Operation	Purpose
<code>addPatient()</code>	Registers a new patient
<code>getPatient()</code>	Retrieves patient information
<code>scheduleAppointment()</code>	Creates an appointment
<code>getMedicalRecords()</code>	Retrieves medical records
<code>updateMedicalRecord()</code>	Updates medical information

5.1 Add Patient

The `addPatient()` operation registers a new patient.

Input

Patient ID
Name
Age
Gender
Phone
Address

Output

Patient ID
Registration Status

Example:

```
<AddPatient>
  <PatientID>P1001</PatientID>
  <Name>Arun Kumar</Name>
  <Age>35</Age>
  <Gender>Male</Gender>
</AddPatient>
```

5.2 Get Patient Records

The `getPatientRecords()` operation retrieves a patient's medical records.

Request:

```
<GetPatientRecords>
  <PatientID>P1001</PatientID>
</GetPatientRecords>
```

Response:

```
<GetPatientRecordsResponse>
  <PatientID>P1001</PatientID>
  <Diagnosis>Fever</Diagnosis>
  <Doctor>Dr. Kumar</Doctor>
  <Treatment>Medication</Treatment>
</GetPatientRecordsResponse>
```

5.3 Schedule Appointment

The appointment operation accepts patient and appointment details.

```
<ScheduleAppointment>
  <PatientID>P1001</PatientID>
  <DoctorID>D205</DoctorID>
  <Date>2026-09-10</Date>
  <Time>10:30</Time>
</ScheduleAppointment>
```

The response may be:

```
<ScheduleAppointmentResponse>
  <AppointmentID>A5001</AppointmentID>
  <Status>Confirmed</Status>
</ScheduleAppointmentResponse>
```

6. Data Exchange Between Client and Server

The communication process occurs as follows:

Step 1 – Client Request

A hospital department sends a SOAP request.

```
Reception System
  ↓
SOAP Request
  ↓
Hospital Web Service
```

Step 2 – Server Validation

The server validates:

- SOAP structure
- Authentication
- Patient information
- Required fields
- XML format

Step 3 – Business Processing

The server executes the requested operation.

For example:

```
Add Patient
  ↓
Validate Patient
  ↓
Store in Database
  ↓
Generate Patient ID
```

Step 4 – SOAP Response

The server sends a SOAP response.

```
Hospital Web Service
  ↓
SOAP Response
  ↓
Reception System
```

7. SOAP Fault Handling

If an error occurs, SOAP provides a **Fault** element.

Example:

```
<soap:Fault>
  <faultcode>soap:Client</faultcode>
  <faultstring>Invalid Patient ID</faultstring>
</soap:Fault>
```

Possible errors include:

- Invalid patient ID
- Missing required information
- Authentication failure
- Database failure
- Invalid XML request

This provides structured error handling between distributed systems.

8. Security of the Hospital System

Hospital systems handle sensitive medical information, so security is important.

The SOAP service should implement:

Authentication

Only authorized hospital applications should access the service.

Authorization

Different users can have different permissions.

For example:

```
Receptionist → Patient registration
Doctor       → Medical records
Billing      → Payment information
Administrator → All authorized operations
```

Encryption

Communication should use secure transport such as HTTPS/TLS to protect data while it is transmitted.

Message Security

Security information can also be included in SOAP headers where required.

9. Why SOAP is Suitable for the Hospital System

SOAP is suitable because it provides:

1. Reliability

SOAP supports structured communication and standardized error handling.

2. Security

SOAP-based systems can use security mechanisms for authentication, authorization, and message protection.

3. Interoperability

Different applications and programming languages can communicate using XML-based SOAP messages.

4. Standardization

SOAP provides a defined message structure using Envelope, Header, and Body.

5. Extensibility

Additional information such as authentication and transaction details can be included in the Header.

Conclusion

A SOAP-based hospital management service provides a structured and interoperable method for exchanging patient, appointment, and medical-record information between hospital departments. Its standardized XML message structure, security mechanisms, and fault handling make it suitable for distributed hospital applications.

QUESTION 2: WSDL Structure for an Enterprise Web Service

1. Introduction

An enterprise application may need to provide services to external systems. The external systems need to know:

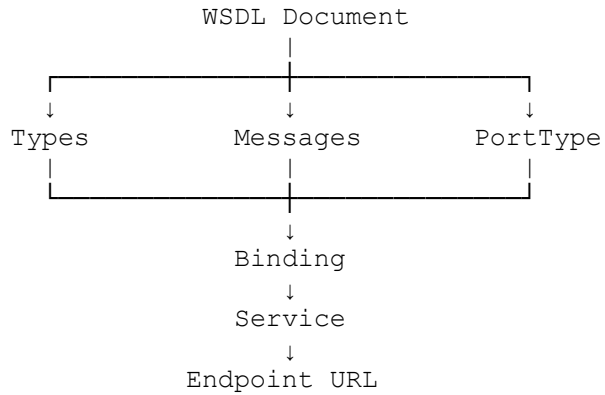
- What operations are available?
- What data should be sent?
- What data will be returned?
- Which communication protocol is used?
- Where is the service located?

WSDL (Web Services Description Language) provides a formal description of a web service.

The question specifically requires the WSDL components:

1. Types
2. Message
3. PortType
4. Binding
5. Service

2. Basic WSDL Architecture



3. Types

The **types** section defines the data types used by the web service.

XML Schema can be used to define complex data structures.

Example:

```
<types>
  <xsd:schema
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">

    <xsd:element name="GetDataRequest">
      <xsd:complexType>
        <xsd:sequence>
          <xsd:element name="id" type="xsd:int"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>

  </xsd:schema>
</types>
```

This defines the structure of the request.

4. Message

The **message** element defines the data exchanged between the client and service.

For example:

```
<message name="GetDataRequest">
  <part name="parameters"
    element="tns:GetDataRequest"/>
```

```
</message>
```

Response:

```
<message name="GetDataResponse">
  <part name="parameters"
    element="tns:GetDataResponse"/>
</message>
```

Messages describe the input and output information.

5. PortType

The **portType** defines the operations provided by the web service.

Example:

```
<portType name="DataServicePortType">

  <operation name="getData">
    <input message="tns:GetDataRequest"/>
    <output message="tns:GetDataResponse"/>
  </operation>

  <operation name="updateData">
    <input message="tns:UpdateDataRequest"/>
    <output message="tns:UpdateDataResponse"/>
  </operation>

</portType>
```

Here, two operations are provided:

- getData
- updateData

The portType acts like the **interface** of the web service.

6. Binding

The **binding** specifies how the operations are communicated.

For example, the service can use SOAP over HTTP.

```
<binding name="DataServiceSoapBinding"
  type="tns:DataServicePortType">
```

```

    <soap:binding
      transport="http://schemas.xmlsoap.org/soap/http"
      style="document"/>

    <operation name="getData">
      <soap:operation soapAction="getData"/>
    </operation>

    <operation name="updateData">
      <soap:operation soapAction="updateData"/>
    </operation>

  </binding>

```

Therefore, the binding connects the abstract service interface with a concrete communication protocol.

7. Service

The **service** element specifies where the web service is available.

Example:

```

<service name="DataService">

  <port name="DataServicePort"
    binding="tns:DataServiceSoapBinding">

    <soap:address
      location="https://example.com/DataService"/>

    </port>

  </service>

```

The service provides the endpoint URL used by the client.

8. Complete WSDL Structure

A simplified WSDL document can be represented as:

```

<?xml version="1.0" encoding="UTF-8"?>

<definitions
  name="DataService"
  targetNamespace="http://example.com/data"
  xmlns:tns="http://example.com/data"
  xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"

```

```

xmlns:xsd="http://www.w3.org/2001/XMLSchema"
xmlns="http://schemas.xmlsoap.org/wsdl/"

<types>

  <xsd:schema
    targetNamespace="http://example.com/data">

    <xsd:element name="GetDataRequest">
      <xsd:complexType>
        <xsd:sequence>
          <xsd:element
            name="id"
            type="xsd:int"/>
        </xsd:sequence>
      </xsd:complexType>
    </xsd:element>

  </xsd:schema>

</types>

<message name="GetDataRequest">
  <part name="parameters"
    element="tns:GetDataRequest"/>
</message>

<message name="GetDataResponse">
  <part name="parameters"
    element="tns:GetDataResponse"/>
</message>

<portType name="DataServicePortType">

  <operation name="getData">
    <input message="tns:GetDataRequest"/>
    <output message="tns:GetDataResponse"/>
  </operation>

</portType>

<binding name="DataServiceBinding"
  type="tns:DataServicePortType">

  <soap:binding
    style="document"
    transport=
      "http://schemas.xmlsoap.org/soap/http"/>

</binding>

<service name="DataService">

  <port
    name="DataServicePort"
    binding="tns:DataServiceBinding">

```

```
        <soap:address
            location=
                "https://example.com/DataService"/>

    </port>

</service>

</definitions>
```

This structure demonstrates how a WSDL document describes the data, messages, operations, communication protocol, and endpoint.

9. How WSDL Defines the Service Interface

The relationship between WSDL components can be understood as:

```
Types
  ↓
Define Data
  ↓
Messages
  ↓
Define Input/Output
  ↓
PortType
  ↓
Define Operations
  ↓
Binding
  ↓
Define Communication Protocol
  ↓
Service
  ↓
Define Endpoint
```

For example:

```
Client
  |
  | Reads WSDL
  ↓
Understands available operations
  |
  ↓
Creates SOAP Request
  |
  ↓
SOAP Web Service
  |
  ↓
```

10. Importance of WSDL in Service Integration

1. Standard Service Description

WSDL provides a standard description of the service.

2. Platform Independence

A client developed using one programming language can communicate with a service developed using another language.

3. Automatic Client Generation

WSDL can be used by development tools to generate client-side service interfaces.

4. Clear Interface Definition

The operations, inputs, outputs, and communication details are clearly specified.

5. Easy Integration

External organizations can understand how to communicate with the service without knowing its internal implementation.

Conclusion

WSDL acts as a formal contract between a service provider and service consumer. The **types, messages, portType, binding, and service** elements collectively describe what the service provides and how clients can communicate with it. Therefore, WSDL is highly important for reliable enterprise service integration.

QUESTION 3: Debugging SOAP Communication Problems

1. Introduction

A company is experiencing problems in its SOAP communication system. Requests are not processed correctly, and responses are delayed or incorrect.

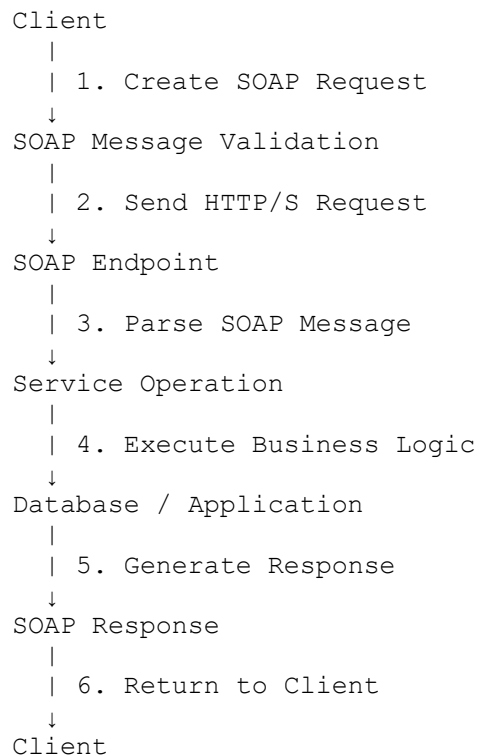
The possible causes include:

- SOAP message formatting errors
- Incorrect namespaces
- Endpoint problems
- Invalid request data
- Server-side errors
- Network problems
- Incorrect SOAPAction
- Authentication failures

The SOAP communication process should therefore be analyzed systematically.

2. SOAP Communication Process

The normal process is:



If an error occurs at any stage, the request may fail or the response may be delayed.

3. Possible Problem 1 – SOAP Message Formatting

A SOAP message must follow the correct XML structure.

Correct structure:

```
<soap:Envelope>
  <soap:Header>
  </soap:Header>

  <soap:Body>
    <GetData>
      <id>1001</id>
    </GetData>
  </soap:Body>
</soap:Envelope>
```

Possible formatting errors include:

- Missing Envelope
- Missing Body
- Incorrect XML closing tags
- Invalid XML characters
- Incorrect element hierarchy
- Missing required fields

Solution

Validate the SOAP XML using an XML parser or XML validation tool before sending it to the server.

4. Possible Problem 2 – Incorrect Namespace

Namespaces are important because SOAP services use namespaces to identify XML elements correctly.

For example:

```
<soap:Envelope
  xmlns:soap=
    "http://schemas.xmlsoap.org/soap/envelope/">
```

If the client uses an incorrect namespace:

```
xmlns:soap="wrong-namespace"
```

the server may not recognize the SOAP message correctly.

Solution

Compare the namespace in the client request with the namespace specified by the WSDL.

5. Possible Problem 3 – Incorrect Endpoint

The client must send the SOAP request to the correct service endpoint.

Example:

```
https://example.com/services/DataService
```

If the URL is incorrect, the request may result in:

```
404 Not Found
```

or connection failure.

Solution

Check the endpoint specified in the WSDL:

```
<soap:address  
  location="https://example.com/services/DataService"/>
```

The client configuration should use the same endpoint.

6. Possible Problem 4 – Incorrect SOAPAction

The SOAP service may expect a specific operation.

For example:

```
<soap:operation soapAction="getData"/>
```

If the client sends the wrong action, the server may not invoke the correct operation.

Solution

Verify the SOAPAction value against the WSDL and service implementation.

7. Possible Problem 5 – Invalid Request Data

The request may contain:

- Wrong data type
- Missing fields
- Invalid ID
- Incorrect date format
- Invalid XML Schema structure

For example, if the service expects:

```
<id>1001</id>
```

but receives:

```
<id>ABC</id>
```

the request may fail because the expected data type is numeric.

Solution

Validate request data against the XML Schema defined by the service.

8. Possible Problem 6 – Authentication and Authorization

The server may reject requests when authentication information is missing or invalid.

Example:

```
<soap:Header>  
  <Security>  
    <Username>user1</Username>  
    <Token>XYZ123</Token>  
  </Security>  
</soap:Header>
```

Solution

Check:

- Username
 - Password/token
 - Security headers
 - User permissions
 - Token expiration
-

9. Possible Problem 7 – Server or Database Problems

The SOAP message may be correct but the server may have an internal problem.

Examples:

```
SOAP Request
  ↓
SOAP Server
  ↓
Database connection failed
  ↓
SOAP Fault
```

Possible causes:

- Database unavailable
- Server overloaded
- Application exception
- Incorrect database query
- Network timeout

Solution

Check server logs and database connectivity.

10. Debugging Approach

A systematic debugging process should be followed.

Step 1 – Validate XML

Check whether the SOAP message is valid XML.

```
SOAP Request
  ↓
XML Validation
  ↓
Valid?
 /      \
No       Yes
 |       |
Fix      Continue
```

Step 2 – Validate SOAP Structure

Check:

- Envelope
- Header
- Body
- Fault

Step 3 – Check Namespace

Compare all namespaces with the WSDL.

Step 4 – Check Endpoint

Verify that the client is sending requests to the correct service URL.

Step 5 – Check SOAPAction

Ensure that the requested operation matches the operation defined in WSDL.

Step 6 – Validate XML Schema

Check that request data matches the expected XML Schema.

Step 7 – Check Authentication

Verify security credentials and authorization.

Step 8 – Check Server Logs

Inspect application logs for:

- Exceptions
- Timeout errors
- Database errors
- Invalid requests

Step 9 – Test Using a SOAP Client

A SOAP testing tool can be used to send controlled requests and inspect responses.

Step 10 – Test Error Handling

Send invalid requests intentionally and verify that the service returns meaningful SOAP Fault messages.

11. SOAP Fault Handling

When a SOAP request cannot be processed, the service should return a structured fault.

Example:

```
<soap:Envelope
  xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">

  <soap:Body>

    <soap:Fault>

      <faultcode>soap:Server</faultcode>

      <faultstring>
        Unable to process request
      </faultstring>

      <detail>
        Database connection failed
      </detail>

    </soap:Fault>

  </soap:Body>

</soap:Envelope>
```

The client can inspect the fault and display an appropriate error or retry the operation when appropriate.

12. Improving Reliability

The corrected SOAP system should implement:

Input Validation

Validate all incoming XML and data types.

Error Handling

Return appropriate SOAP Fault messages.

Logging

Record requests, responses, errors, and processing times where appropriate.

Timeout Handling

Configure reasonable connection and response timeouts.

Authentication

Verify authorized users and applications.

Secure Communication

Use HTTPS/TLS to protect data during transmission.

Monitoring

Monitor:

- Request success rate
 - Response time
 - Server availability
 - Error frequency
-

13. Before-and-After Communication

Before Debugging

```
Client
↓
Incorrect SOAP Request
↓
Server
↓
✗ Namespace / XML / Endpoint Error
↓
Incorrect or delayed response
```

After Debugging

```
Client
↓
Validated SOAP Request
↓
Correct Namespace
↓
Correct Endpoint
↓
Authentication
↓
Service Processing
```


↓
Database
↓
Validated SOAP Response
↓
Client

This produces more reliable communication.

14. Debugging Checklist

Area	Check
XML	Is the XML well formed?
Envelope	Is the SOAP Envelope correct?
Header	Is security information valid?
Body	Does it contain the correct operation?
Namespace	Does it match WSDL?
SOAPAction	Is the operation correct?
Endpoint	Is the service URL correct?
Schema	Does data match XML Schema?
Authentication	Are credentials valid?
Server	Are there application errors?
Database	Is the database available?
Response	Is the SOAP response valid?
Fault	Are errors properly reported?

15. Conclusion

The SOAP communication problem can be resolved through systematic validation of the SOAP message, namespaces, endpoint, SOAPAction, XML Schema, authentication, server logs, and database connectivity.

Once these components are correctly configured, the communication process becomes:

Validated Request
↓
Correct Endpoint
↓
Authenticated Service
↓
Business Processing
↓

Validated Response



Reliable Client Communication

Therefore, proper SOAP validation, structured error handling, logging, security, and monitoring are essential for achieving reliable communication in distributed enterprise systems.